Raspberry Pi Zero 2 W Camera Server

Technology overview, comparison with alternatives, and future work

Date: 26 October 2026

Document version: 1.1 (matches server.py version 1.1)

Target: Raspberry Pi Zero 2 W, 32-bit Raspberry Pi OS Lite (Bookworm), camera-only node

Contents

1. Introduction and goals 2

2. System architecture 2

How the main mechanisms work . . . 3

3. Technologies used 3

Why raw H.264 over HTTP instead of MJPEG, RTSP or WebRTC . . . 4

4. Comparison with other server options 4

4.1 aiohttp compared with Flask and FastAPI . . . 4

4.2 Python server compared with a C++ server . . . 5

5. V4L2 (Video4Linux2) 5

6. GStreamer 6

7. Approaches at a glance 6

8. Current limitations 6

9. Future work (topics only, not implemented) 7

9.1 Running the server as a service . . . 7

9.2 Disabling Bluetooth . . . 7

9.3 Wi-Fi power saving and link stability . . . 7

9.4 Trimming the operating system for final deployment . . . 8

9.5 Other future topics . . . 8

10. Conclusion 9

1. Introduction and goals

This document describes the camera server that runs on a Raspberry Pi Zero 2 W. The Pi acts as a camera-only node: it exposes its camera over HTTP so that another Raspberry Pi, a Jetson or a laptop can request still images, watch live video, or request a recording of a chosen length. The document explains the technology used, compares it with common alternatives (Flask, FastAPI, a C++ server, V4L2 and GStreamer), and lists topics for future work.

> **Scope note:** the Future work section (section 9) lists topics only. None of those items has been implemented yet. Performance figures in this document are typical or expected values and have not yet been measured on the target device.

Requirements

- One endpoint lists 5 image tiers, 4 video tiers and 4 timed-recording tiers, each ordered from highest to lowest quality.
- One endpoint serves images, one serves live video, and one serves a recording of t seconds, where t is supplied by the client.
- A home page documents every endpoint with examples.
- Minimal footprint: the Pi only serves its camera and runs no other models or tasks.
- Constraints of the board: 512 MB RAM, quad-core 1 GHz CPU, one 2.4 GHz Wi-Fi radio and one hardware H.264 encoder.

Endpoints

Endpoint	Purpose	Response
`GET /`	Documentation home page with examples (public, no token)	text/html
`GET /options`	Lists all image, video and record tiers with ready-to-use URLs	application/json
`GET /image?q=i1..i5`	Takes one photo at the chosen tier	image/jpeg
`GET /video?q=v1..v4`	Live stream; stays open until the client disconnects	video/h264 (raw, chunked)
`GET /record?q=r1..r4&t=SECONDS`	Recording of t seconds (maximum 60), then the response ends	video/h264 (raw)

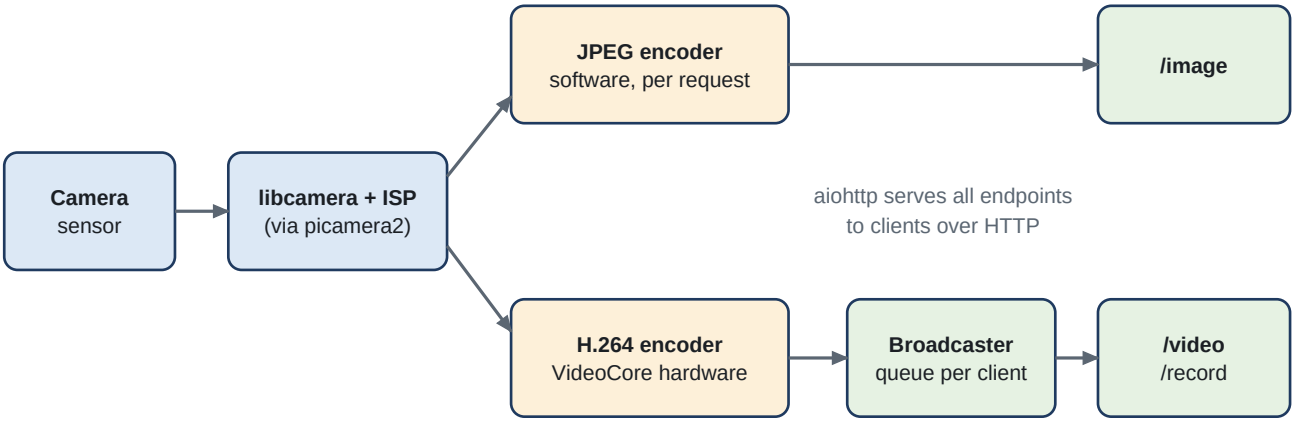
Default tiers (assuming a Camera Module v2; tiers larger than the sensor are removed automatically)

Level	Image tier	Video and record tier
1 (highest)	i1: 3280x2464, JPEG quality 95	v1 / r1: 1920x1080, 30 fps, 6 Mbps
2	i2: 1920x1080, quality 90	v2 / r2: 1280x720, 30 fps, 3 Mbps
3	i3: 1280x720, quality 85	v3 / r3: 640x480, 25 fps, 1 Mbps
4	i4: 640x480, quality 80	v4 / r4: 320x240, 15 fps, 0.3 Mbps
5 (lowest)	i5: 320x240, quality 70	-

2. System architecture

One Python process owns the camera and serves every endpoint. Stills and video take different paths through the camera stack, but both are controlled by one camera manager so that the two never compete for the device.

Still image path



Live video and recording path

Figure 1. Data paths inside the camera server.

How the main mechanisms work

- **Single camera owner.** Only one process can use the camera at a time, so one process holds it. A lock serialises changes between still capture and video, and the manager records the current mode.
- **Hardware H.264 encoding.** Video frames go from the ISP to the VideoCore hardware encoder, so compression uses almost no CPU. Python only forwards finished, encoded chunks.
- **Fan-out broadcaster.** The encoder writes into one broadcaster object that copies every frame into a queue per connected client. Several viewers cost little more than one.
- **Keyframe synchronisation.** A new viewer waits for the next keyframe (about one second) so that decoding starts cleanly. A viewer that falls behind has its backlog dropped and rejoins at the next keyframe, so it never slows the others.
- **Profile locking.** The encoder has a fixed resolution, frame rate and bitrate. Clients on the same tier share it. A request for a different tier while someone is watching receives HTTP 409.
- **Timed recordings.** The record endpoint joins the stream at a keyframe, forwards data for t seconds and then ends the response. It uses the same hardware-encoded stream as live video.
- **On-demand camera.** The camera starts on the first request and stops 8 seconds after the last client leaves, which saves power and heat.
- **Still images.** The camera is configured for the requested resolution, auto-exposure is given about one second to settle on a cold start, and the JPEG is encoded in software.
- **Access control.** An optional token is accepted as an Authorization: Bearer header or a token query parameter. The home page is public and contains no secrets.

3. Technologies used

Component	Role in this project	Why it was chosen
Raspberry Pi Zero 2 W	Deployment node with the camera attached	Small and low power. Quad-core Cortex-A53 at 1 GHz, 512 MB RAM, CSI camera connector, hardware H.264 encoder, built-in Wi-Fi.
Raspberry Pi OS Lite, 32-bit (Bookworm)	Operating system	No desktop. The 32-bit build uses less memory than 64-bit on a 512 MB board, and libcamera and picamera2 support it.
libcamera	Camera framework and ISP pipeline	Provides demosaicing, auto exposure and auto white balance. It replaces the legacy camera stack.

Component	Role in this project	Why it was chosen
picamera2	Python interface to libcamera	Handles camera configuration, still capture, the hardware H.264 encoder and output objects, in far less code than a native implementation.
VideoCore H.264 hardware encoder	Video compression	Compresses 1080p30 without loading the CPU. Accessed through a V4L2 memory-to-memory device.
Python 3 (system, via apt)	Application language	Installed from the OS packages, so no pip or virtual environment is needed on the device.
asyncio and aiohttp	HTTP server	One process, event-loop based, with cheap long-lived streaming responses and few dependencies.
Raw H.264 (Annex-B) over chunked HTTP	Video transport	No muxer needed on the Pi. Plays in ffplay, VLC, GStreamer and OpenCV. Recordings are wrapped into MP4 on the client without re-encoding.
JPEG (libjpeg-turbo based)	Still image encoding	Software encoding in picamera2 is fast enough for single photos.
Plain HTML, CSS and JavaScript	Home page	Served by the same process. No extra packages and no external resources.

Why raw H.264 over HTTP instead of MJPEG, RTSP or WebRTC

- **MJPEG** is simple and browser friendly, but needs a software JPEG encode for every frame and uses far more bandwidth than H.264, which is a problem on a 2.4 GHz Wi-Fi link and a small CPU.
- **RTSP and WebRTC** are better for many viewers and for browsers, but need an extra server (for example MediaMTX) or additional libraries on the Pi.
- **Raw H.264 over HTTP** adds almost no code, uses the hardware encoder directly, and can be consumed with standard tools. Its main drawbacks are that browsers cannot play it directly and that the stream has no timestamps or container.

4. Comparison with other server options

4.1 aiohttp compared with Flask and FastAPI

Criterion	Flask	FastAPI	aiohttp (used)
Concurrency model	WSGI, synchronous; one thread or worker per open stream	ASGI, asyncio, run by uvicorn	asyncio native, built-in server
Many long-lived streaming clients	Fine for a few viewers; each stream occupies a thread, which is costly on 512 MB	Efficient with async generators and StreamingResponse	Efficient with StreamResponse
Packages on the Pi	Flask plus a production server such as waitress or gunicorn	FastAPI, Starlette, pydantic and uvicorn	One apt package with a few small dependencies
Validation and API docs	Manual	Built in (typed parameters, OpenAPI page)	Manual (a few lines for q and t); the home page serves as documentation
Handing frames from the encoder thread	Thread-safe queues	call_soon_threadsafe into the event loop	call_soon_threadsafe into the event loop
Expected footprint	Moderate	Highest of the three	Lowest of the three
Best fit	Prototypes with one to three viewers	Larger APIs where typed schemas and automatic docs matter	Few endpoints, streaming focus, minimal packages

The framework has little influence on video quality or frame rate, because compression is done by the hardware encoder. It mainly affects memory use, the number of installed packages and how many simultaneous viewers the Pi can serve comfortably. FastAPI also works on a Zero 2 W and is a reasonable choice if automatic validation and generated API documentation become important.

4.2 Python server compared with a C++ server

Criterion	Python (picamera2 + aiohttp)	C++ (libcamera + V4L2 + HTTP library)
Development effort	Low: roughly 300 lines using picamera2	High: camera, encoder, buffers, HTTP and JPEG all written or integrated by hand (rpicam-apps is a useful reference)
Camera and encoder access	picamera2 wraps libcamera, the V4L2 encoder and JPEG encoding	libcamera C++ API, V4L2 memory-to-memory device for H.264, libjpeg-turbo for stills
CPU load for this workload	Low: heavy work happens in hardware and C libraries	Low: marginally lower, the difference is small
Memory	Interpreter, picamera2 and NumPy add tens of MB (to be measured)	A few MB beyond libcamera
Start-up time	Several seconds for imports on this CPU	Near instant
On-device frame processing (future)	Per-frame Python and NumPy overhead can matter	Better suited
Safety and maintenance	Easy to change; memory safe	Needs care with threads and buffer lifetimes
Deployment	apt packages only	One compiled binary, but build needs libcamera development packages (build elsewhere or cross-compile)
HTTP libraries	aiohttp	cpp-httplib, Boost.Beast, Crow or libmicrohttpd

Assessment. The expensive work (ISP processing, H.264 compression, JPEG for single photos) already runs in compiled code or dedicated hardware, and the real limits are Wi-Fi throughput and the single hardware encoder, neither of which depends on the language. A C++ server is justified if measurements show that RAM or start-up time is a problem, or if frame analysis will run on the Pi itself. If a port is ever needed, the practical route is to reuse the libcamera C++ API and the V4L2 codec device and to keep the same HTTP interface, so clients are unaffected.

5. V4L2 (Video4Linux2)

V4L2 is the Linux kernel interface for video devices. Cameras, video decoders and encoders appear as device nodes (for example /dev/video0) and are controlled with ioctl calls and memory-mapped buffers.

Role on the Raspberry Pi

- With the current libcamera stack, the camera sensor interface and the ISP are exposed as V4L2 devices, but they deliver raw sensor data and need libcamera to run the ISP pipeline (exposure, white balance, demosaicing). A plain V4L2 application would see unprocessed frames.
- The hardware H.264 encoder is a V4L2 memory-to-memory device: raw frames are queued in, encoded frames come out. picamera2 and rpicam-vid already use it, so this project uses V4L2 indirectly.
- The older "camera as a webcam" V4L2 mode belongs to the legacy camera stack, which is deprecated.

Using V4L2 directly

- **Advantages:** full control of buffers and encoder settings, minimal overhead, no Python runtime.
- **Disadvantages:** a lot of low-level code (buffer queues, format negotiation, keyframe handling) and you must still use libcamera for a usable image. Behaviour also varies between kernel versions.

- **Conclusion:** V4L2 is the right layer only when writing a native C or C++ implementation. For this project picamera2 already provides the same encoder access with far less code.

6. GStreamer

GStreamer is a multimedia framework in which media flows through a pipeline of plugins (sources, encoders, parsers, muxers and sinks). It is a legitimate alternative for the live-video part of this server.

Typical pipeline on the Pi (illustrative, not part of the current system)

```
libcamerasrc ! video/x-raw,width=1280,height=720,framerate=30/1
  ! v4l2h264enc extra-controls="controls,video_bitrate=3000000"
  ! h264parse ! mpegtsmux ! (network sink)
```

Advantages

- Mature muxing and streaming: MPEG-TS, MP4, HLS and RTSP (through gst-rtsp-server).
- Built-in fan-out with the tee element, and many protocols and sinks ready to use.
- Symmetric on the consumer side: a Jetson can decode with nvv4l2decoder in the same framework.

Disadvantages on a Zero 2 W

- Heavier footprint: more libraries and plugins load into 512 MB of RAM.
- Changing the quality tier means tearing down and rebuilding the pipeline, which is awkward to do from a request handler.
- One-shot stills with per-request resolution are clumsy; picamera2 handles them natively.
- Caps negotiation with the hardware encoder can be fiddly and varies by GStreamer and libcamera version.
- An HTTP layer is still required for the options, image and record endpoints, so GStreamer would not replace the web server.

Assessment. Keep picamera2 for the whole server now. Consider GStreamer, or MediaMTX for the live video only, if RTSP, WebRTC, HLS or many simultaneous viewers become requirements.

7. Approaches at a glance

Approach	Effort	Footprint on Zero 2 W	Flexibility	Best when
picamera2 + aiohttp (this project)	Low	Moderate	High: custom endpoints and tiers	A custom API with a few viewers
picamera2 + Flask or FastAPI	Low	Moderate to higher	High	You prefer those frameworks or need generated API docs
C++ with libcamera + V4L2	High	Lowest	Highest	RAM or start-up time is critical, or frames are processed on the Pi
GStreamer pipeline + small HTTP layer	Medium	Higher (many plugins)	High for streaming, lower for stills	RTSP, HLS or many viewers are needed
rpicam-vid / rpicam-still + MediaMTX	Low	Low to moderate	Low: fixed feature set	A quick RTSP or WebRTC stream with no custom API

8. Current limitations

- Only one live tier runs at a time. A request for a different tier while someone is watching gets HTTP 409.

- The image endpoint is unavailable (HTTP 409) while a video stream is active. A dual-stream design could remove this.
- The raw H.264 stream has no timestamps or container. Recordings must be wrapped on the client using the tier's real frame rate.
- Traffic is plain HTTP, so the token travels in clear text. The server is intended for a trusted network.
- The Zero 2 W has 2.4 GHz Wi-Fi only. Real throughput is often around 15 to 25 Mbps, so the highest tiers may not be sustainable on a weak link.
- The design relies on the VideoCore hardware H.264 encoder found on the Zero 2 W and earlier Pi models. The Pi 5 has no hardware H.264 encoder, so the video path would need a software encoder there.
- CPU load, RAM use, temperature and real throughput have not yet been measured on the target hardware.

9. Future work (topics only, not implemented)

This section lists the topics to be investigated and implemented in later stages. No changes described here have been made to the current system.

9.1 Running the server as a service

- Unit design: start after the network is online, run as an unprivileged user in the video group, and restart on failure with a short delay.
- Secrets: keep the access token out of the unit file, in an environment file with restricted permissions.
- Boot behaviour: enable at boot, and check start-up ordering against Wi-Fi and camera readiness.
- Logging: use the system journal, set a sensible log level and limit log volume to protect the SD card.
- Graceful stop: confirm that a stop request triggers the clean shutdown path so the camera is released.
- Safeguards: restart rate limiting, optional memory limit, and a watchdog to restart a hung process.
- Deployment: a simple, repeatable way to install a new version and to roll back.

Expected benefit: the server starts at power-up, recovers from crashes and no longer needs manual process handling.

9.2 Disabling Bluetooth

- What to turn off: the Bluetooth service and its UART attach service, and optionally the device-tree overlay that disables the Bluetooth hardware at boot.
- Trade-offs: small savings in idle CPU, memory and power, and the UART is freed. On the Zero 2 W the Wi-Fi and Bluetooth share one radio chip, so Wi-Fi stability must be re-checked after the change.
- Verification: confirm the services are inactive, no Bluetooth controller is present, and compare idle resource use before and after.
- Recovery: document how to re-enable Bluetooth in case it is needed for maintenance.

Expected benefit: a slightly lighter idle system and fewer background components.

9.3 Wi-Fi power saving and link stability

- Power-save mode: it can add latency and cause stutter in streams. Topics are how to turn it off persistently with the network manager and how to check its current state.
- Trade-off: slightly higher power draw, which matters only if the Pi runs from a battery.
- Link quality: signal strength, 2.4 GHz channel choice, access point placement and the regulatory country setting.

- Resilience: behaviour after a link loss, a fixed address through a DHCP reservation, and reliability of the hostname.local name.
- Measurement: test throughput and latency for each tier and record the real sustainable bitrate, then adjust the tier table.

Expected benefit: smoother video and tier choices based on measured link capacity.

9.4 Trimming the operating system for final deployment

Graphical interface and desktop

- Preferred route: build new deployments from Raspberry Pi OS Lite, which has no desktop.
- If a desktop image was used: switch the default boot target to console only, then remove the desktop environment, display manager, window manager, web browser and office or education applications together with their automatically installed dependencies.

Unneeded services and packages

- Candidates: printing support, modem manager, hotkey daemon, sound packages, man pages and documentation, extra locales, compilers and development tools that are not needed on the device, and Bluetooth packages.
- Items to decide on: the mDNS service (useful for hostname.local) and the rpicam tools (useful for diagnostics).
- Never remove: libcamera and everything picamera2 depends on, the network manager, SSH and the Python packages the server uses.

Hardware features that are not used

- HDMI output, audio output, status LEDs, and unused interfaces such as I2C, SPI and the serial console, controlled through boot configuration options and overlays.

Fewer SD card writes and a faster boot

- Journal in volatile or size-limited storage, swap file policy, disabling automatic update timers, and reduced file-access-time updates.
- Longer term: a read-only root filesystem or overlay, with attention to filesystem checks after power loss.
- Boot analysis with the systemd tools to find slow or unnecessary units.

Safe procedure

- Back up the SD card image before any removal.
- Simulate each removal first to see which dependencies would also go.
- Remove in small steps and re-test the camera server after each step and after a reboot.
- Keep a written list of what was removed, and a recovery path (second SD card or serial console).
- Alternative: build a custom minimal image so the deployed system is reproducible instead of trimmed by hand.

Measurements to record before and after

- Number of installed packages, disk usage, free RAM at idle, time from power-on to the first served request, idle CPU temperature and power draw.

Expected benefit: more free RAM and storage, faster boot, a smaller attack surface and less SD card wear.

9.5 Other future topics

- Transport security: TLS, or a VPN at the router, instead of plain HTTP.
- Snapshots during live streams through a dual-stream configuration.
- MediaMTX in front of the live video for RTSP, WebRTC or HLS, and MP4 output straight from the Pi.

- Camera controls (exposure, white balance, focus) exposed through the API.
- A configuration file for the tier tables, a health and metrics endpoint, and automated tests and benchmarks on real hardware.

10. Conclusion

For a camera-only Raspberry Pi Zero 2 W, the combination of picamera2, the hardware H.264 encoder and an asyncio-based aiohttp server gives the required four endpoints with the least code and the fewest installed packages. Flask and FastAPI would work but add threads or dependencies for no gain in video quality. A C++ server, direct V4L2 use or a GStreamer pipeline become attractive only if measurements show a RAM or start-up limit, if analysis moves onto the Pi, or if RTSP and many viewers are required. The next steps are to measure the system on the device and then work through the future topics in section 9.

Further reading

- Picamera2 manual (Raspberry Pi) and the Raspberry Pi camera software documentation (rpicam-apps).
- libcamera project documentation.
- aiohttp documentation: server and streaming responses.
- Linux kernel documentation: V4L2 and the memory-to-memory codec interface.
- GStreamer documentation: libcamerasrc, v4l2h264enc and gst-rtsp-server.
- MediaMTX documentation.